МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей

Кафедра ПОИТ

ОТЧЕТ

по лабораторной работе № 1

по дисциплине «Методы машинного обучения»

на тему: «Основы Python»

Вариант № 3

Выполнил(а):
студент(ка) гр. 351002
Беседин А.А.

Проверил:
Преподаватель
Лебедевич А.В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью лабораторной работы является освоение базовых возможностей языка программирования Python, применяемого в дальнейшем для решения задач машинного обучения: изучение встроенных типов данных и коллекций, управляющих конструкций, функций и механизма обработки исключений, а также принципов объектно-ориентированного программирования (инкапсуляции, наследования и полиморфизма). Практическая часть работы заключается в разработке и тестировании набора консольных программ, решающих задачи индивидуального задания.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Язык Python и базовые типы данных

Python – высокоуровневый интерпретируемый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью. Благодаря простоте синтаксиса и развитой экосистеме библиотек (NumPy, pandas, scikit-learn, PyTorch) Python является основным языком в области анализа данных и машинного обучения.

Любое значение в Python является объектом, а переменная – лишь ссылкой на объект. Тип переменной определяется во время выполнения программы по значению, которое ей присвоено. К основным встроенным типам относятся целые числа `int` (произвольной точности), вещественные числа `float`, логический тип `bool` со значениями `True` и `False` и строки `str`.

Строка – неизменяемая упорядоченная последовательность символов Юникода. Строки поддерживают индексацию `s[i]`, срезы `s[start:stop:step]` (например, `s[::-1]` возвращает перевернутую строку) и большое число методов. В работе используются методы проверки символов: `isdigit()` (цифра), `isalpha()` (буква), `isupper()` и `islower()` (прописная и строчная буква), а также `upper()` и `split()`.

Ввод данных с клавиатуры выполняется функцией `input()`, которая всегда возвращает строку, поэтому для получения чисел используется явное преобразование типов `int()` и `float()`. Вывод выполняется функцией `print()`. Функция `map()` применяет заданное преобразование к каждому элементу последовательности, что позволяет компактно прочитать ряд чисел: `list(map(int, input().split()))`.

2.2 Коллекции

Список `list` – изменяемая упорядоченная коллекция элементов произвольных типов. Списки поддерживают добавление элементов (`append`), подсчет вхождений (`count`), поиск индекса первого вхождения (`index`, в том числе с заданной начальной позицией) и срезы.

Кортеж `tuple` – неизменяемая упорядоченная коллекция. Кортежи применяются для хранения фиксированных наборов значений и для возврата нескольких результатов из функции: запись `return even, odd` фактически возвращает кортеж, который затем распаковывается в несколько переменных.

Словарь `dict` – изменяемое отображение уникальных хешируемых ключей на значения, реализованное в виде хеш-таблицы. Доступ к значению по ключу выполняется в среднем за время $O(1)$. Обход словаря в цикле `for` перебирает ключи, метод `items()` возвращает пары «ключ – значе-

ние», а оператор `in` проверяет наличие ключа. Начиная с версии 3.7 словари сохраняют порядок вставки элементов.

2.3 Управляющие конструкции и функции

Ветвление реализуется конструкцией `if – elif – else`. Для организации циклов служат оператор `for`, перебирающий элементы любого итерируемого объекта (в том числе диапазона `range(n)`), и оператор `while`, выполняющий тело, пока истинно условие. Оператор `break` досрочно завершает цикл, `continue` – переходит к следующей итерации.

Функция объявляется ключевым словом `def`, принимает параметры и возвращает результат оператором `return`. Функции позволяют выделить логически завершенные фрагменты алгоритма и использовать их повторно. Встроенная функция `isinstance(obj, cls)` проверяет принадлежность объекта к типу, что позволяет реализовать функцию, поведение которой зависит от типа аргумента.

Примером алгоритма, использующего цикл `while`, является проверка числа на простоту. Натуральное число $n \geq 2$ простое, если оно не имеет делителей d, удовлетворяющих условию

$$2 \leq d \leq \sqrt{n}, \tag{1}$$

поэтому перебор делителей достаточно вести до $\sqrt{n}$, что дает сложность алгоритма $O(\sqrt{n})$.

2.4 Обработка исключений

Исключение – объект, сигнализирующий об ошибке во время выполнения программы. Если исключение не обработано, выполнение программы прерывается. Для перехвата исключений используется конструкция `try – except – else – finally`: в блоке `try` размещается потенциально опасный код, блоки `except` перехватывают исключения указанных типов, блок `else` выполняется при отсутствии ошибок, а блок `finally` выполняется всегда, независимо от того, возникло ли исключение. Типичными встроенными исключениями являются `ValueError` (некорректное значение, например, при преобразовании строки «abc» в число) и `ZeroDivisionError` (деление на ноль).

2.5 Объектно-ориентированное программирование

Класс – пользовательский тип данных, описывающий состояние (атрибуты) и поведение (методы) объектов. Объект (экземпляр класса) создается вызовом класса, при этом автоматически вызывается метод-инициализатор

`__init__`, в котором задаются атрибуты экземпляра. Первый параметр метода `self` – ссылка на текущий объект.

Инкапсуляция – объединение данных и методов в классе и ограничение прямого доступа к внутреннему состоянию. В Python она реализуется соглашениями об именовании: одно подчеркивание (`_area`) обозначает защищенный атрибут, два подчеркивания (`__make_deal`) – приватный, имя которого искажается интерпретатором (name mangling) и недоступно извне класса напрямую.

Наследование позволяет создать производный класс на основе базового, переняв его атрибуты и методы. Вызов `super().__init__()` передает управление инициализатору родительского класса. Полиморфизм – возможность единообразно работать с объектами разных классов через общий интерфейс: производные классы переопределяют метод базового класса, и при вызове выполняется реализация, соответствующая фактическому типу объекта.

Атрибуты класса, объявленные в теле класса, разделяются всеми экземплярами. Методы класса, помеченные декоратором `@classmethod`, получают первым параметром сам класс `cls` и применяются для работы с атрибутами класса. Статические методы (`@staticmethod`) не получают ни экземпляра, ни класса и представляют собой обычные функции, логически сгруппированные внутри класса.

В качестве примера предметной логики класса рассмотрим треугольник со сторонами a, b, c. Треугольник существует, если выполняются неравенства

$$a + b > c, \quad a + c > b, \quad b + c > a, \tag{2}$$

а его площадь вычисляется по формуле Герона

$$S = \sqrt{p\,(p-a)(p-b)(p-c)}, \quad p = \frac{a+b+c}{2}, \tag{3}$$

где S – площадь треугольника;
p – полупериметр треугольника;
a, b, c – длины сторон треугольника.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

В соответствии с вариантом № 3 требуется разработать на языке Python набор консольных программ, состоящий из трех частей.

3.1 Часть 1. Базовые конструкции и коллекции

1. Определить, сколько в числе четных цифр, а сколько нечетных. Число вводится с клавиатуры. – 2 балла
2. Посчитать, сколько пар (стоят рядом) верхнего и нижнего регистра находится в веденном с клавиатуры слове. (Пример HjkLM- 1 пара нижнего, 1 пара верхнего), а также сколько всего букв в слове. – 2 балла
3. Найдите сумму отрицательных элементов списка. Найдите сумму элементов списка между двумя первыми нулями. Если двух нулей нет в списке, то выведите ноль. – 1 балл
4. В настольной игре Скрабл (Scrabble) каждая буква имеет определенную ценность. В случае с английским алфавитом очки распределяются так: A, E, I, O, U, L, N, S, T, R – 1 очко; D, G – 2 очка; B, C, M, P – 3 очка; F, H, V, W, Y – 4 очка; K – 5 очков; J, X – 6 очков; Q, Z – 10 очков.
   Напишите программу, которая вычисляет стоимость введенного пользователем слова. – 2 балла
5. Реализуйте программу «Магазин автозапчастей», которая будет включать в себя шесть пунктов меню. У вас есть словарь, где ключ – название продукции. Значение – список, который содержит состав продукции, цену и кол-во (шт),которое есть в магазине. 1. Просмотр описания: название – описание 2. Просмотр цены: название – цена. 3. Просмотр количества: название – количество. 4. Всю информацию. 5. Покупка В пункте «Покупка» необходимо совершить покупку, с клавиатуры вводите название продукции и его кол-во, n – выход из программы. Посчитать цену выбранных товаров и сколько товаров осталось в изначальном списке 6. До свидания – 2 балла
6. В кортеже целых чисел найдите максимальный и минимальный элементы. – 1 балл

3.2 Часть 2. Функции и обработка исключений

1. Напишите функцию is_password_good(password), которая принимает в качестве аргумента строковое значение пароля password и возвращает значение True если пароль является надежным и False в противном случае. Пароль является надежным если: – его длина не менее 8 символов; – он содержит как минимум одну заглавную букву (верхний регистр); – он содержит хотя бы одну цифру. – 2 балла

2. Напишите функцию, которая будет принимать один аргумент. Если в функцию передаётся словарь, то вывести ключ с максимальным значением. Если список, то найти количество элементов, расположенных до первого отрицательного элемента. Число – проверить простое, или нет. Строка – вывести в обратном порядке. Найти сумму цифр в строке. Сделать проверку со всеми этими случаями. – 4 балла
3. Даны два числа n и m. Создайте двумерный массив размером n×m и заполните его символами "." и "*" в шахматном порядке. В левом верхнем углу должна стоять точка. – 2 балла
4. Напишите программу, демонстрирующую работу try\except\finally – 2 балла

3.3 Часть 3. Объектно-ориентированное программирование

Задание 1: Класс Triangle Создайте класс Triangle. В нём пропишите 3 (метода) функции. Первый метод: проверка на существование треугольника по данным сторонам. Второй метод : нахождение площади треугольника. Третий метод: нахождение периметра треугольника. – 1 балл

Задание 2: Класс House 1. Создайте класс House 2. Создайте метод __init__() и определите внутри него два динамических свойства: _area и _price. Свои начальные значения они получают из параметров метода __init__() 3. Создайте метод final_price(), который принимает в качестве параметра размер скидки и возвращает цену с учетом данной скидки. Класс SmallHouse 1. Создайте класс SmallHouse, унаследовав его функционал от класса House 2. Внутри класса SmallHouse переопределите метод __init__() так, чтобы он создавал объект с площадью 40м2 Класс Human 1. Реализуйте приватный метод make_deal(), который будет отвечать за техническую реализацию покупки дома: уменьшать количество денег на счету и присваивать ссылку на только что купленный дом. В качестве аргументов данный метод принимает объект дома и его цену. 2. Реализуйте метод buy_house(), который будет проверять, что у человека достаточно денег для покупки, и совершать сделку. Если денег слишком мало - нужно вывести предупреждение в консоль. Параметры метода: ссылка на дом и размер скидки – 3 балла

Задание 3: Создать классы Circle (круг), Square (квадрат), Rectangle (прямоугольник) для описания плоских геометрических фигур. Реализовать метод нахождения площади фигуры. Переопределить метод нахождения площади фигуры.

Задание 4:

Придумать класс самостоятельно, реализовать в нем методы экземпляра класса, статические, методы, методы класса. – 1 – 3 балла

4 КОД РЕШЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Ниже приведен полный исходный код программ, разработанных для решения индивидуального задания.

4.1 Часть 1. Базовые конструкции и коллекции

4.1.1 Задание 1

```
def count_even_odd(number):
    even = 0
    odd = 0
    for d in number:
        if d.isdigit():
            if int(d) % 2 == 0:
                even += 1
            else:
                odd += 1
    return even, odd


def main():
    number = input("Enter a number: ")
    even, odd = count_even_odd(number)
    print("Even digits:", even)
    print("Odd digits:", odd)


main()
```

4.1.2 Задание 2

```
def count_pairs(word):
    upper_pairs = 0
    lower_pairs = 0
    for i in range(len(word) - 1):
        if word[i].isupper() and word[i + 1].isupper():
            upper_pairs += 1
        elif word[i].islower() and word[i + 1].islower():
            lower_pairs += 1
    return upper_pairs, lower_pairs


def count_letters(word):
    letters = 0
```

```
    for ch in word:
        if ch.isalpha():
            letters += 1
    return letters


def main():
    word = input("Enter a word: ")
    upper_pairs, lower_pairs = count_pairs(word)
    print("Lowercase pairs:", lower_pairs)
    print("Uppercase pairs:", upper_pairs)
    print("Total letters:", count_letters(word))


main()
```

4.1.3 Задание 3

```
def sum_negative(numbers):
    total = 0
    for x in numbers:
        if x < 0:
            total += x
    return total


def sum_between_zeros(numbers):
    if numbers.count(0) < 2:
        return 0
    first = numbers.index(0)
    second = numbers.index(0, first + 1)
    return sum(numbers[first + 1:second])


def main():
    numbers = list(map(int, input("Enter numbers separated
  by spaces: ").split()))
    print("Sum of negative elements:", sum_negative(numbers)
  )
    print("Sum between the first two zeros:",
  sum_between_zeros(numbers))


main()
```

4.1.4 Задание 4

```
points = {
    "AEIOULNSTR": 1,
    "DG": 2,
    "BCMP": 3,
    "FHVWY": 4,
    "K": 5,
    "JX": 6,
    "QZ": 10,
}


def word_score(word):
    total = 0
    for ch in word.upper():
        for letters, value in points.items():
            if ch in letters:
                total += value
    return total


def main():
    word = input("Enter a word: ")
    print("Word score:", word_score(word))


main()
```

4.1.5 Задание 5

```
products = {
    "filter": ["oil filter, steel and paper", 450, 20],
    "pads": ["brake pads, steel and friction lining", 2300,
  12],
    "spark plug": ["spark plug, ceramic and iridium", 600,
  40],
    "battery": ["car battery, lead and electrolyte", 7800,
  5],
    "belt": ["timing belt, reinforced rubber", 1900, 8],
}


def show_description():
    for name in products:
        print(name, "-", products[name][0])
```

```
def show_price():
    for name in products:
        print(name, "-", products[name][1])


def show_count():
    for name in products:
        print(name, "-", products[name][2])


def show_all():
    for name in products:
        print(name, "-", products[name][0], "- price:",
  products[name][1], "- quantity:", products[name][2])


def buy():
    total = 0
    while True:
        name = input("Product name (n - exit): ")
        if name == "n":
            break
        if name not in products:
            print("No such product")
            continue
        count = int(input("Quantity: "))
        if count > products[name][2]:
            print("Not enough in stock, available:",
  products[name][2])
            continue
        products[name][2] -= count
        total += products[name][1] * count
    print("Purchase total:", total)
    print("Left in the shop:")
    show_count()


def main():
    while True:
        print("\n1. View descriptions")
        print("2. View prices")
        print("3. View quantities")
        print("4. All information")
        print("5. Purchase")
```

```
        print("6. Goodbye")
        choice = input("Choose an option: ")
        if choice == "1":
            show_description()
        elif choice == "2":
            show_price()
        elif choice == "3":
            show_count()
        elif choice == "4":
            show_all()
        elif choice == "5":
            buy()
        elif choice == "6":
            print("Goodbye!")
            break
        else:
            print("Invalid option")


main()
```

4.1.6 Задание 6

```
def find_max(numbers):
    result = numbers[0]
    for x in numbers:
        if x > result:
            result = x
    return result


def find_min(numbers):
    result = numbers[0]
    for x in numbers:
        if x < result:
            result = x
    return result


def main():
    numbers = tuple(map(int, input("Enter numbers separated
  by spaces: ").split()))
    print("Maximum element:", find_max(numbers))
    print("Minimum element:", find_min(numbers))
```

```
main()
```

4.2 Часть 2. Функции и обработка исключений

4.2.1 Задание 1

```
def is_password_good(password):
    if len(password) < 8:
        return False
    has_upper = False
    has_digit = False
    for c in password:
        if c.isupper():
            has_upper = True
        if c.isdigit():
            has_digit = True
    return has_upper and has_digit


def main():
    password = input("Enter a password: ")
    print("Password is good:", is_password_good(password))


main()
```

4.2.2 Задание 2

```
def max_key(d):
    best = None
    for key in d:
        if best is None or d[key] > d[best]:
            best = key
    return best


def count_before_negative(numbers):
    count = 0
    for n in numbers:
        if n < 0:
            break
        count += 1
    return count


def is_prime(n):
```

```
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True


def digit_sum(s):
    total = 0
    for c in s:
        if c.isdigit():
            total += int(c)
    return total


def process(value):
    if isinstance(value, dict):
        print("Key with max value:", max_key(value))
    elif isinstance(value, list):
        print("Elements before first negative:",
  count_before_negative(value))
    elif isinstance(value, int):
        print("Is prime:", is_prime(value))
    elif isinstance(value, str):
        print("Reversed:", value[::-1])
        print("Sum of digits:", digit_sum(value))


def main():
    pairs = input("Enter dictionary as key:value pairs
  separated by spaces: ").split()
    d = {}
    for pair in pairs:
        key, val = pair.split(":")
        d[key] = int(val)
    process(d)

    numbers = list(map(int, input("Enter list of numbers: ")
  .split()))
    process(numbers)

    number = int(input("Enter a number: "))
    process(number)
```

```
    s = input("Enter a string: ")
    process(s)


main()
```

4.2.3 Задание 3

```
def make_board(n, m):
    board = []
    for i in range(n):
        row = []
        for j in range(m):
            if (i + j) % 2 == 0:
                row.append(".")
            else:
                row.append("*")
        board.append(row)
    return board


def main():
    n = int(input("Enter n: "))
    m = int(input("Enter m: "))
    board = make_board(n, m)
    for row in board:
        print(" ".join(row))


main()
```

4.2.4 Задание 4

```
def divide(a, b):
    try:
        result = int(a) / int(b)
        print("Result:", result)
    except ValueError:
        print("Error: please enter integers")
    except ZeroDivisionError:
        print("Error: division by zero")
    finally:
        print("Finally block executed")
```

```
def main():
    a = input("Enter first number: ")
    b = input("Enter second number: ")
    divide(a, b)


main()
```

4.3 Часть 3. Объектно-ориентированное программирование

4.3.1 Задание 1

```
import math


class Triangle:
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def exists(self):
        a = self.a
        b = self.b
        c = self.c
        return a > 0 and b > 0 and c > 0 and a + b > c and a
    + c > b and b + c > a

    def area(self):
        p = self.perimeter() / 2
        return math.sqrt(p * (p - self.a) * (p - self.b) * (
   p - self.c))

    def perimeter(self):
        return self.a + self.b + self.c


def main():
    a, b, c = map(float, input("Enter three sides: ").split
   ())
    triangle = Triangle(a, b, c)
    if triangle.exists():
        print("Triangle exists")
        print("Area:", triangle.area())
        print("Perimeter:", triangle.perimeter())
    else:
```

```
        print("Triangle does not exist")


main()
```

4.3.2 Задание 2

```
class House:
    def __init__(self, area, price):
        self._area = area
        self._price = price

    def final_price(self, discount):
        return self._price * (100 - discount) / 100


class SmallHouse(House):
    def __init__(self, price):
        super().__init__(40, price)


class Human:
    def __init__(self, name, money):
        self.name = name
        self.money = money
        self.house = None

    def __make_deal(self, house, price):
        self.money -= price
        self.house = house

    def buy_house(self, house, discount):
        price = house.final_price(discount)
        if self.money < price:
            print("Not enough money to buy the house")
        else:
            self.__make_deal(house, price)
            print("House bought for", price)


def main():
    name = input("Enter your name: ")
    money = float(input("Enter your money: "))
    price = float(input("Enter small house price: "))
    discount = float(input("Enter discount in percent: "))
    human = Human(name, money)
```

```
    house = SmallHouse(price)
    human.buy_house(house, discount)
    print("Money left:", human.money)
    print("Has house:", human.house is not None)


main()
```

4.3.3 Задание 3

```
import math


class Figure:
    def area(self):
        return 0


class Circle(Figure):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2


class Square(Figure):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2


class Rectangle(Figure):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height


def main():
    radius = float(input("Enter circle radius: "))
    side = float(input("Enter square side: "))
```

```
    width, height = map(float, input("Enter rectangle width
  and height: ").split())
    figures = [Circle(radius), Square(side), Rectangle(width
  , height)]
    for figure in figures:
        print(type(figure).__name__, "area:", figure.area())


main()
```

4.3.4 Задание 4

```
class BankAccount:
    interest_rate = 5
    accounts_count = 0

    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance
        BankAccount.accounts_count += 1

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Not enough money")
        else:
            self.balance -= amount

    def add_interest(self):
        self.balance += self.balance * BankAccount.
  interest_rate / 100

    @classmethod
    def set_interest_rate(cls, rate):
        cls.interest_rate = rate

    @classmethod
    def get_accounts_count(cls):
        return cls.accounts_count

    @staticmethod
    def is_valid_amount(amount):
        return amount > 0
```

```
def main():
    owner = input("Enter owner name: ")
    balance = float(input("Enter start balance: "))
    account = BankAccount(owner, balance)

    amount = float(input("Enter deposit amount: "))
    if BankAccount.is_valid_amount(amount):
        account.deposit(amount)
    else:
        print("Invalid amount")

    amount = float(input("Enter withdraw amount: "))
    if BankAccount.is_valid_amount(amount):
        account.withdraw(amount)
    else:
        print("Invalid amount")

    rate = float(input("Enter interest rate: "))
    BankAccount.set_interest_rate(rate)
    account.add_interest()

    print("Owner:", account.owner)
    print("Balance:", account.balance)
    print("Accounts count:", BankAccount.get_accounts_count
   ())


main()
```

5 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Каждая программа была запущена из командной строки на нескольких наборах входных данных, включающих как типичные, так и граничные случаи.

5.1 Часть 1. Базовые конструкции и коллекции

Результат работы программы подсчета четных и нечетных цифр приведен на рисунке 1. Для числа 4815162342 найдено 6 четных и 4 нечетные цифры, для числа 13579 четные цифры отсутствуют.

```
Windows PowerShell

PS D:\University\7Sem\MMO\lab1\sublab1\task1> python main.py
Enter a number: 4815162342
Even digits: 6
Odd digits: 4
PS D:\University\7Sem\MMO\lab1\sublab1\task1> python main.py
Enter a number: 13579
Even digits: 0
Odd digits: 5
PS D:\University\7Sem\MMO\lab1\sublab1\task1>
```

Рисунок 1 – Подсчет четных и нечетных цифр числа

На рисунке 2 показан подсчет пар соседних строчных и прописных букв и общего количества букв в слове.

```
Windows PowerShell

PS D:\University\7Sem\MMO\lab1\sublab1\task2> python main.py
Enter a word: HeLLoWORLDpython
Lowercase pairs: 5
Uppercase pairs: 5
Total letters: 16
PS D:\University\7Sem\MMO\lab1\sublab1\task2> python main.py
Enter a word: MachineLEARNING
Lowercase pairs: 5
Uppercase pairs: 7
Total letters: 15
PS D:\University\7Sem\MMO\lab1\sublab1\task2>
```

Рисунок 2 – Подсчет пар строчных и прописных букв в слове

Вычисление суммы отрицательных элементов и суммы элементов между первыми двумя нулями приведено на рисунке 3. Во втором запуске нулевые элементы отсутствуют, поэтому вторая сумма равна нулю.

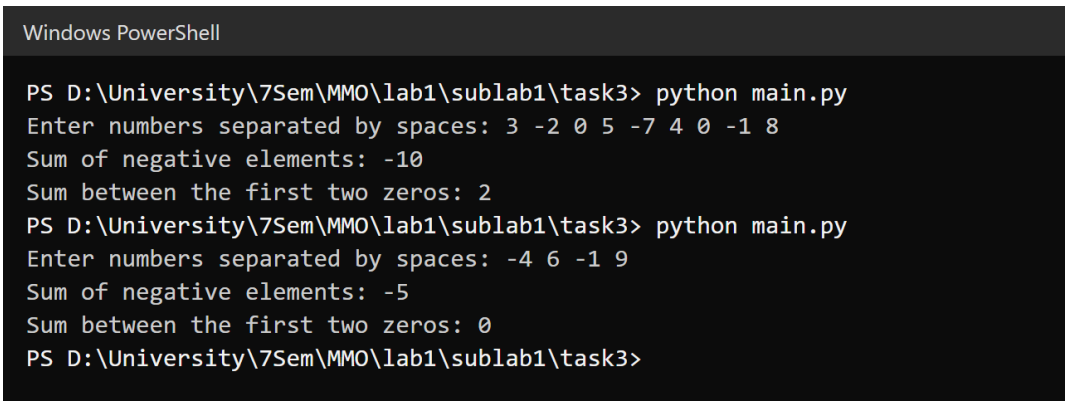


Рисунок 3 – Вычисление сумм элементов списка

На рисунке 4 приведен подсчет стоимости слов: слово Quiz оценено в $10 + 1 + 1 + 10 = 22$ очка.

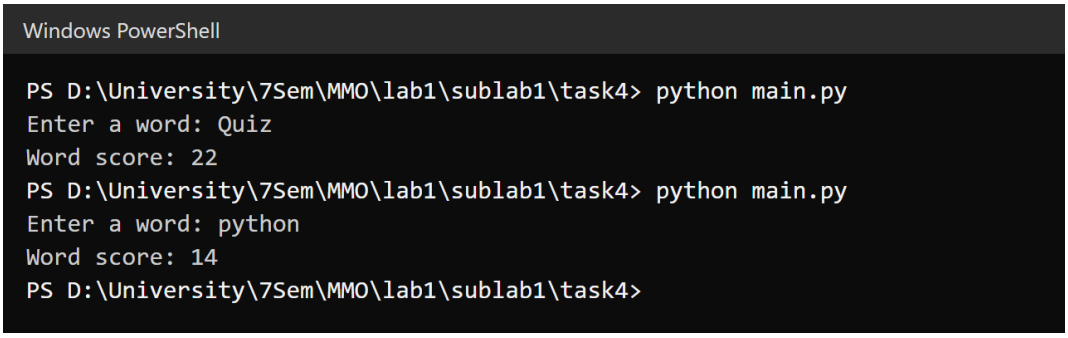


Рисунок 4 – Подсчет стоимости слова по правилам игры «Эрудит»

Работа пунктов меню программы «Магазин автозапчастей», отвечающих за просмотр информации о товарах, а также реакция на некорректный пункт меню показаны на рисунке 5.

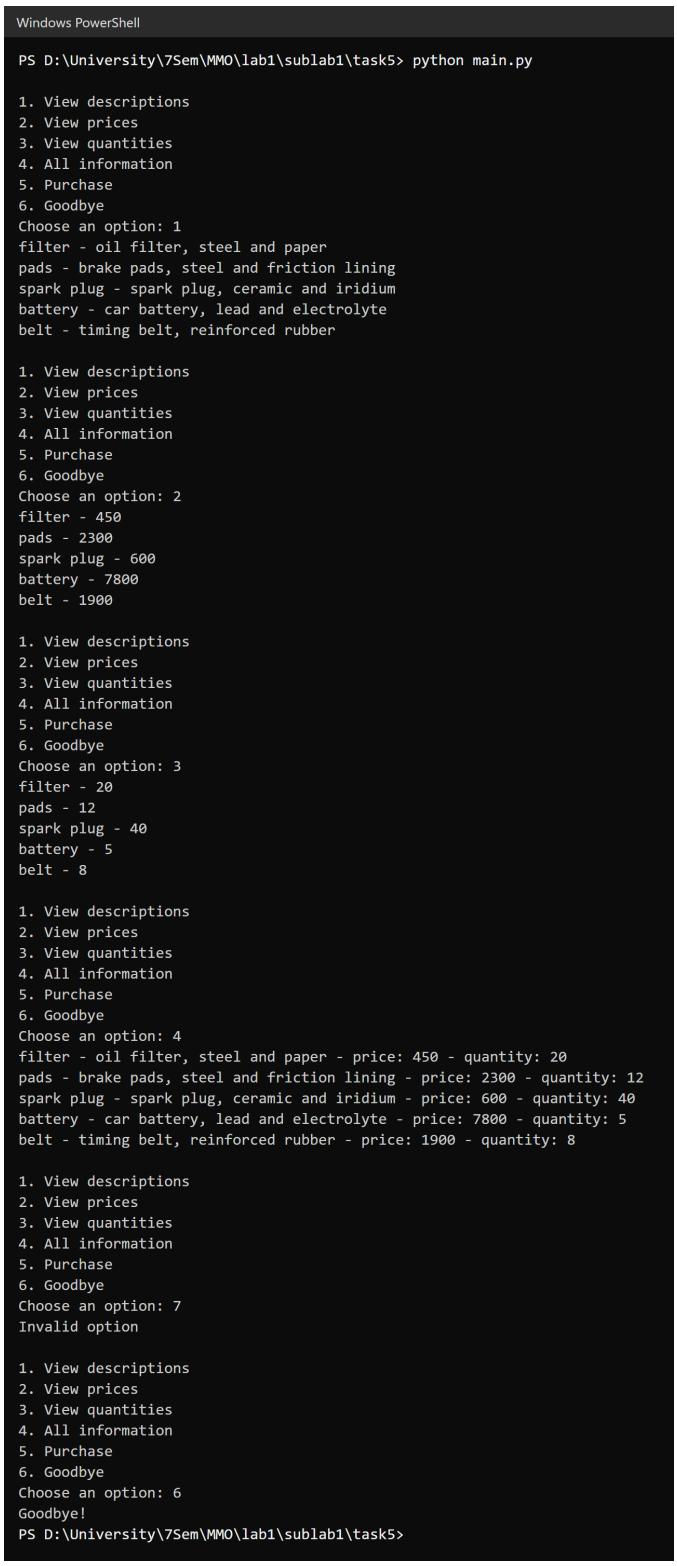


Рисунок 5 – Просмотр информации о товарах в программе «Магазин автозапчастей»

Процесс покупки товаров приведен на рисунке 6. Программа отклоняет покупку при недостаточном количестве товара на складе и при вводе несуществующего названия, подсчитывает итоговую сумму покупки ($2 \cdot 2300 + 2 \cdot 7800 = 20200$) и выводит обновленные остатки.

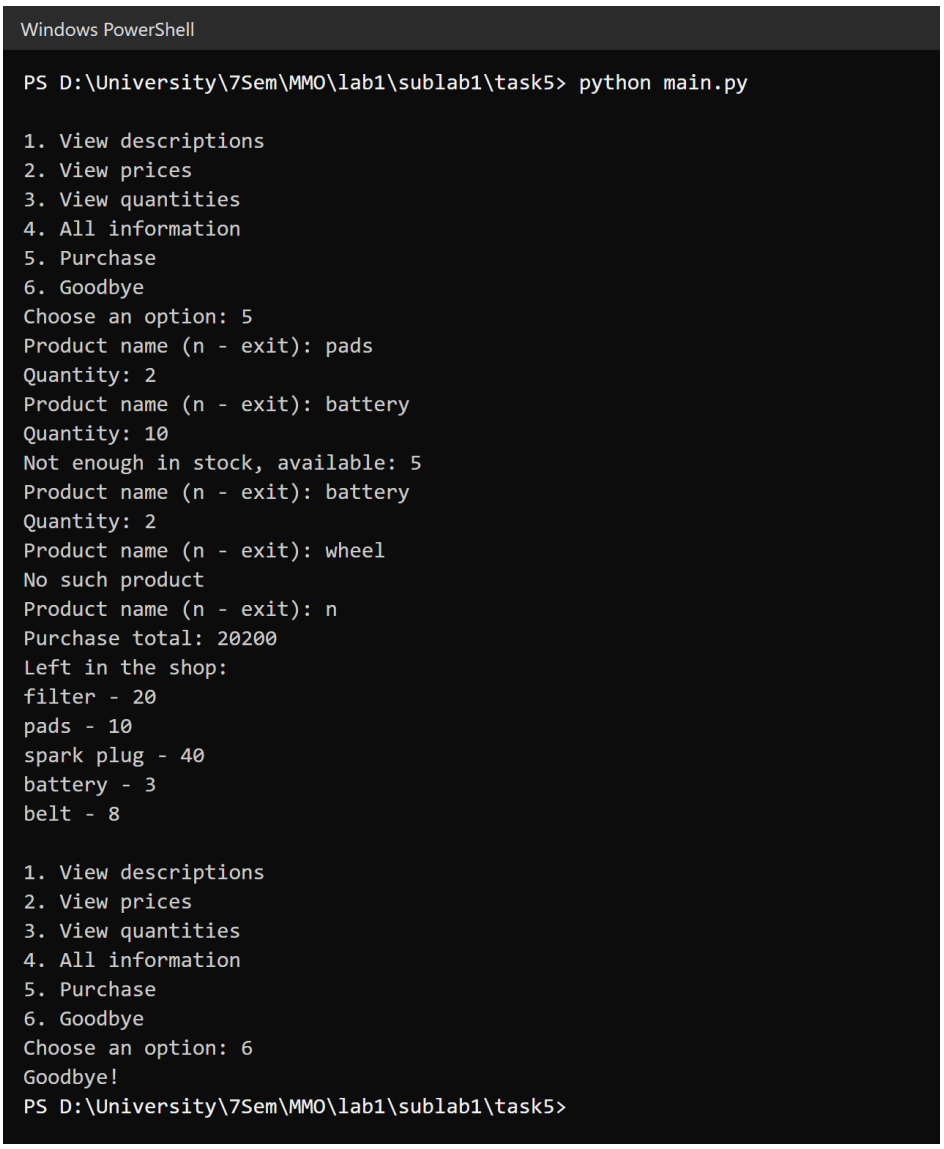


Рисунок 6 – Покупка товаров в программе «Магазин автозапчастей»

Поиск максимального и минимального элементов кортежа показан на рисунке 7.

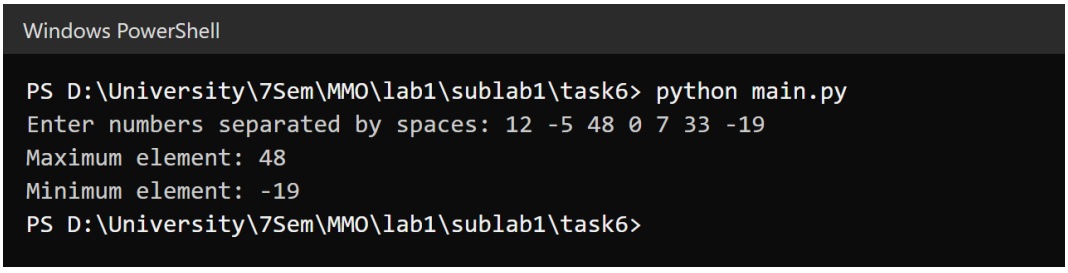


Рисунок 7 – Поиск максимального и минимального элементов кортежа

5.2 Часть 2. Функции и обработка исключений

Проверка надежности паролей приведена на рисунке 8: пароль qwerty слишком короткий, пароль password123 не содержит прописных букв, и только

пароль Password123 удовлетворяет всем требованиям.

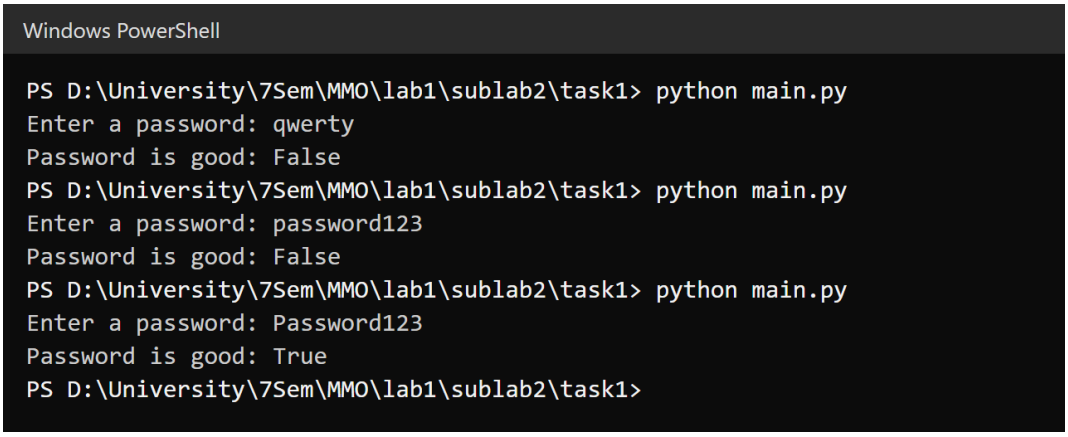


Рисунок 8 – Проверка надежности паролей

На рисунке 9 показана обработка аргументов различных типов одной функцией: словаря, списка, целого числа и строки.

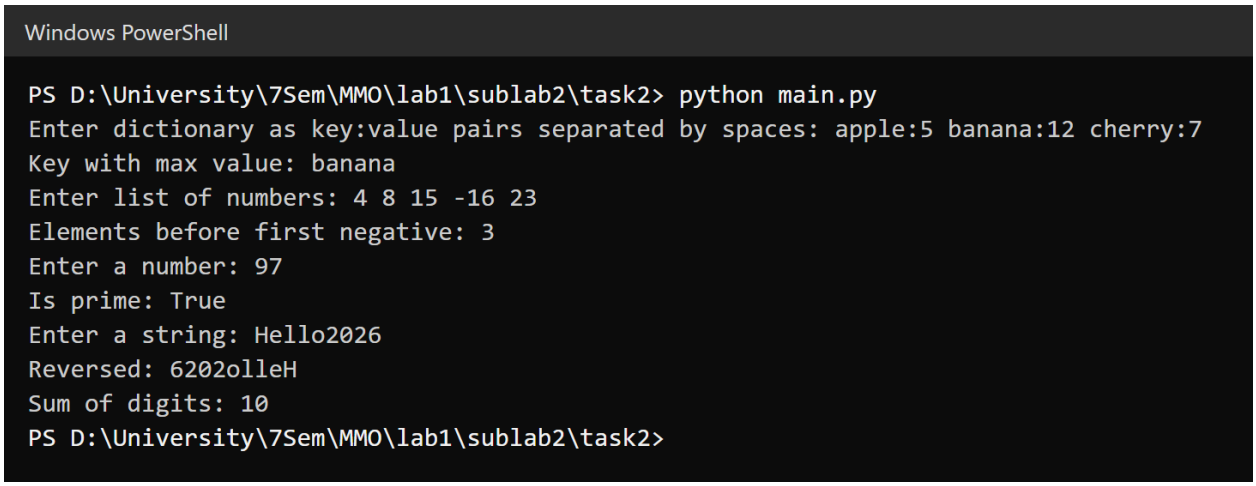


Рисунок 9 – Обработка аргументов различных типов

Вывод шахматных досок размером 5×8 и 3×3 приведен на рисунке 10.

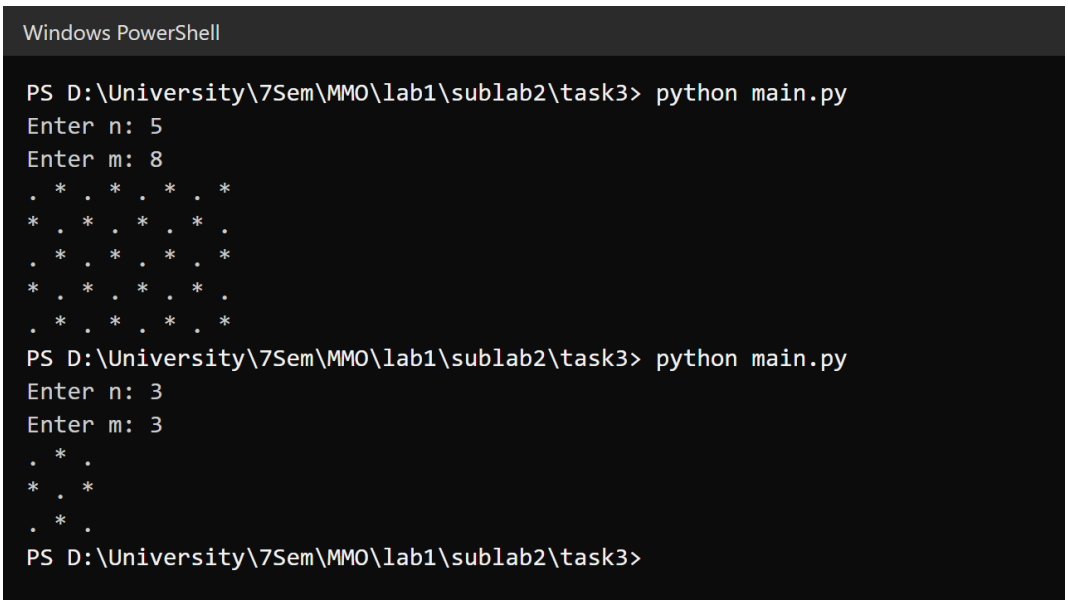


Рисунок 10 – Формирование шахматной доски

Обработка исключений при делении чисел показана на рисунке 11: корректное деление, деление на ноль и ввод нечислового значения. Во всех трех случаях выполняется блок `finally`.

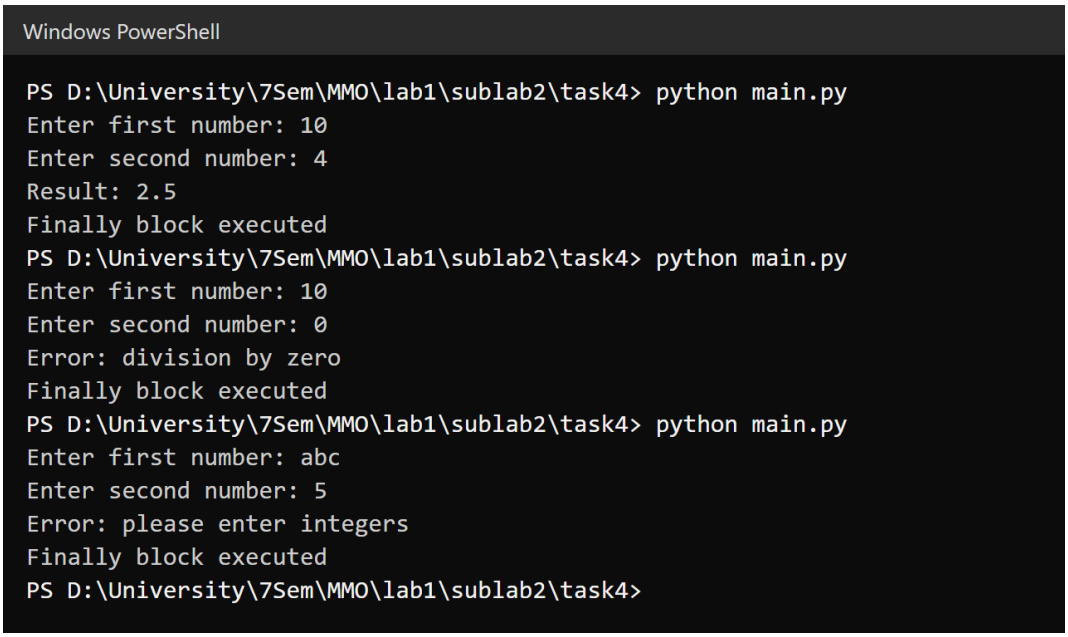


Рисунок 11 – Обработка исключений при делении чисел

5.3 Часть 3. Объектно-ориентированное программирование

На рисунке 12 приведена работа класса `Triangle`: для сторон 3, 4, 5 вычислены площадь и периметр, а для сторон 1, 2, 10 определено, что треугольник не существует.

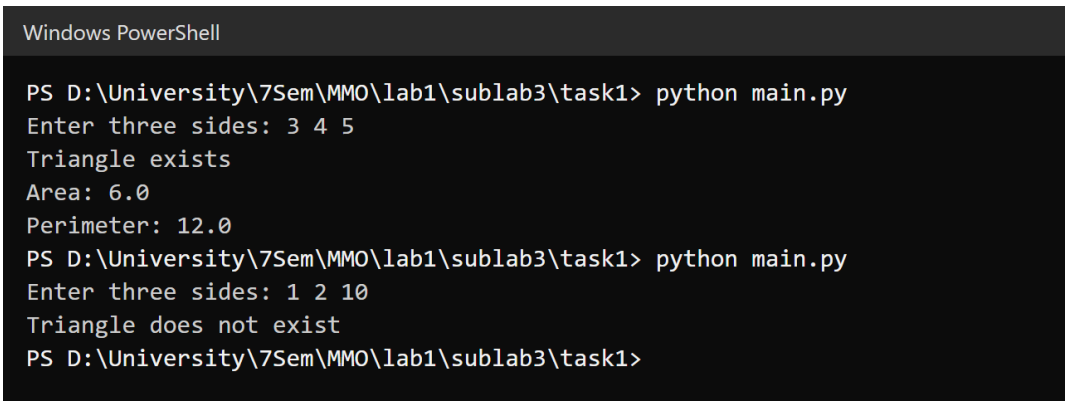


Рисунок 12 – Работа класса Triangle

Покупка дома объектом класса `Human` показана на рисунке 13. В первом случае денег достаточно для покупки дома с учетом скидки 10 %, во втором – покупка отклоняется.

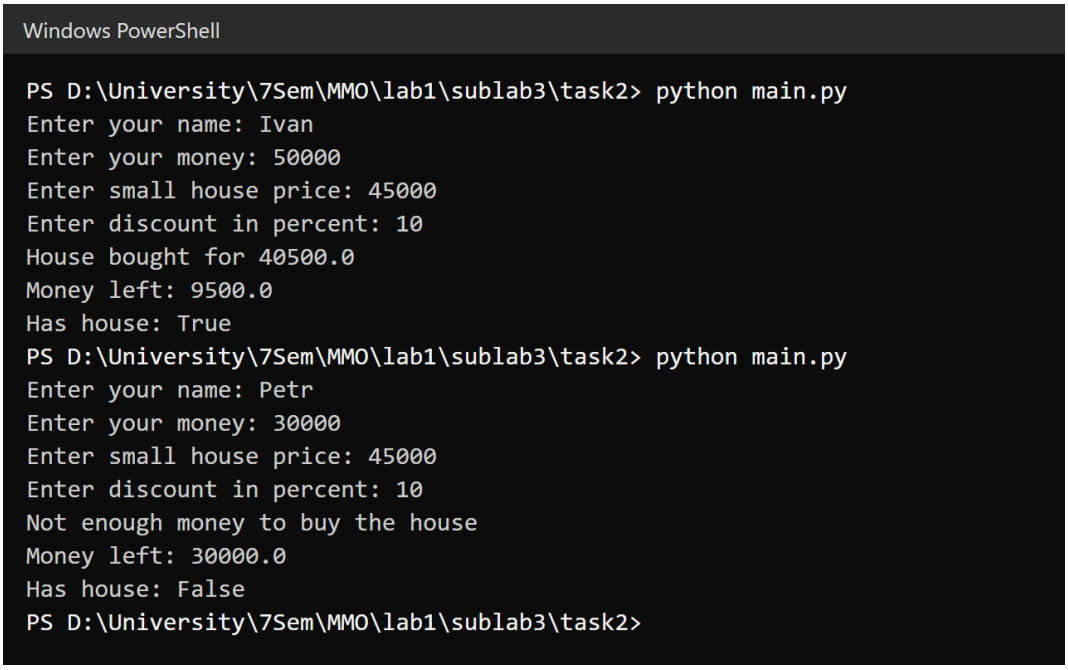


Рисунок 13 – Покупка дома объектом класса Human

Вычисление площадей фигур через единый метод `area()` базового класса приведено на рисунке 14.

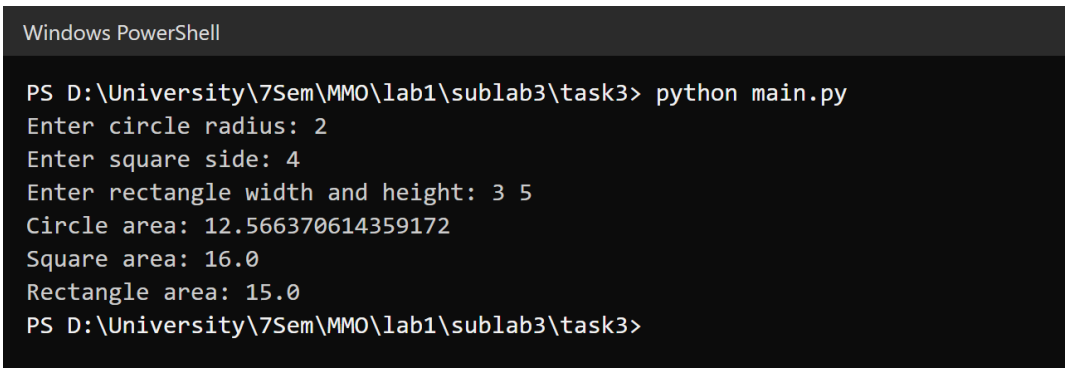


Рисунок 14 – Демонстрация полиморфизма при вычислении площадей фигур

Работа класса `BankAccount` показана на рисунке 15. В первом запуске после пополнения на 500 и снятия 200 на баланс 1300 начислено 10 %, во втором – статический метод отклоняет отрицательную сумму пополнения, а снятие суммы, превышающей баланс, запрещается.

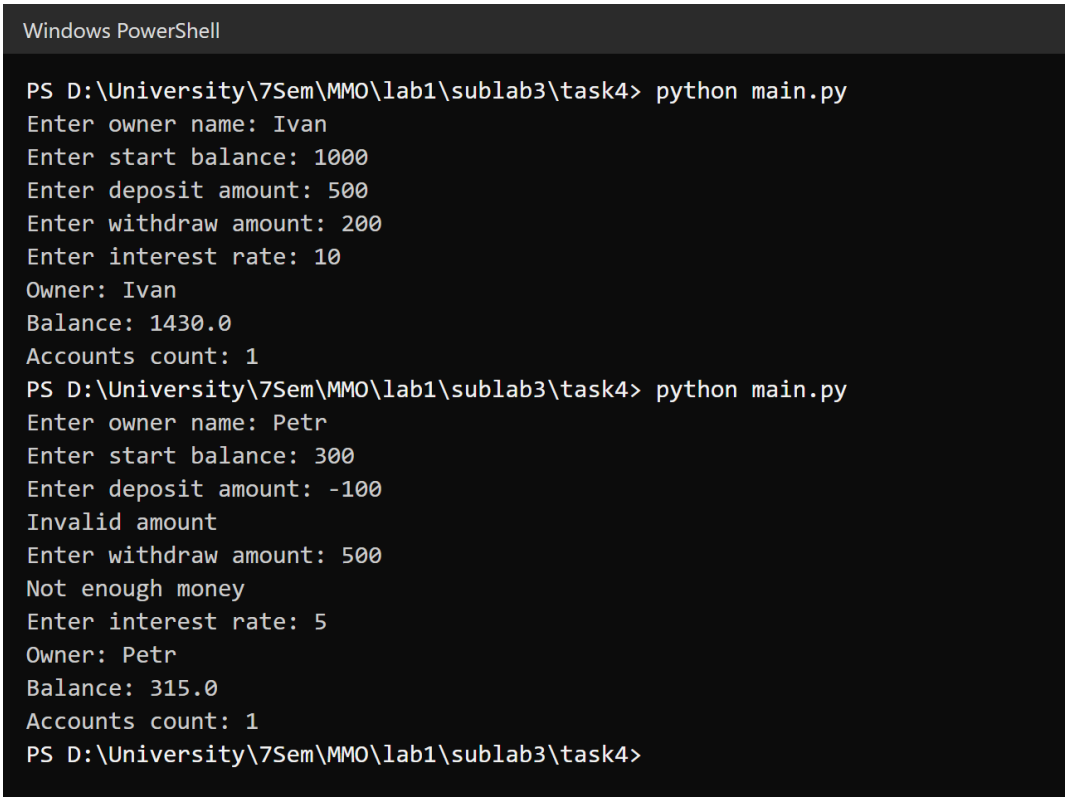


Рисунок 15 – Работа класса BankAccount

6 ВЫВОДЫ

В ходе выполнения лабораторной работы № 1 были получены следующие результаты:

1) Изучены встроенные типы данных и коллекции языка Python (строки, списки, кортежи, словари), а также основные управляющие конструкции. На их основе разработаны программы обработки чисел, строк и списков и интерактивная программа «Магазин автозапчастей» с меню.
2) Освоены объявление и использование функций, возврат нескольких значений и проверка типа аргумента с помощью `isinstance`. Изучен механизм обработки исключений `try – except – finally`, позволяющий программе корректно реагировать на ошибочный ввод и деление на ноль.
3) Изучены принципы объектно-ориентированного программирования: разработаны классы, демонстрирующие инкапсуляцию (защищенные и приватные члены), наследование, полиморфизм, а также использование атрибутов класса, методов класса и статических методов.
4) Все программы протестированы на типичных и граничных наборах входных данных, результаты их работы совпадают с ожидаемыми. Полученные навыки программирования на Python будут использованы в последующих лабораторных работах по методам машинного обучения.